Dokumentace k semestrálnímu projektu z předmětu ZPG

Souhlasím s vystavením této semestrální práce na stránkách katedry informatiky a výpočetní techniky a jejímu využití pro prezentaci pracoviště.

Jméno: Oldřich Jan Švehla
E-mail: tmwmf@gapps.zcu.cz
Datum: 26. května 2025
Studijní číslo: A23B0234P

Řešení kolizí

Kolizní systém řeší interakce hráče s prostředím na několika úrovních:

- **Horizontální kolize hráče:** Hráč má kruhový hitbox definovaný vlastností *radius* ve třídě *Camera*. Při pokusu o pohyb metodou *Camera.TryMove* se detekuje průnik tohoto hitboxu s Axis-Aligned Bounding Box (AABB) kolizních objektů (*Model.CollisionBox* typu AABB). K detekci se využívá metoda *AABB.playerIntersect*.
- **Klouzání podél překážek:** Pokud je detekována kolize v zamýšleném směru pohybu, systém provede dodatečný test pohybu zvlášť po ose X a Z. To umožňuje hráči plynule "klouzat" podél stěn, aniž by se úplně zastavil.
- **Vertikální kolize hráče:** Pohyb hráče ve vertikálním směru (gravitace, skoky, dopady) spravuje třída *PlayerController*. Metoda *PlayerController.CheckCollisionWithObjects* vytváří pro hráče AABB a testuje jeho průnik s AABB kolizních modelů ve světě (*Model.CollisionBox.Intersects*). Tím se řeší dopady na podlahu, objekty nebo detekce stropu.
- **Kolizní objekty:** Každý objekt Model ve scéně má vlastnost *IsCollidable* a může mít definovaný *CollisionBox* (AABB). AABB pro modely se typicky vytváří v *World.CreateModel* na základě rozměrů modelu.

Implementované funkcionality

- Zobrazení mapy načtené ze souboru jako síť 3D modelů dle typu objektu
- Podlaha a strop pomocí samostatných modelů
- Vícenásobné světelné zdroje na stropě, dynamická baterka následující hráče
- Detekce kolizí hráče se zdmi a objekty
- Kamera s volným pohledem a ovládáním myší (WASD + myš)
- Možnost rotace objektů (např. *collectible* se otáčí kolem své osy)
- Načítání objektů a světel dle znaku ze souboru mapy (např. *, o, x...)
- Vícepatrové bludiště s možností se propadnout o patro níže skrze definované díry (*WorldElement.Hole*)
- Minimapa zobrazující aktuální patro, pozici a orientaci hráče, včetně průlezů a dalších prvků mapy.
- Portály (Teleporty) umožňující přesun hráče mezi definovanými body na mapě s vizuálním efektem přechodu.

Rozšíření

Projekt byl připraven na rozšíření o vícenásobné světelné zdroje. Každý objekt typu *Light* generuje model lampy a instanci světla, která je uložená ve scéně. Zatím však *shader* zpracovává pouze jeden aktivní světelný zdroj (např. baterku hráče). Funkční podpora více světel bude doplněna v další fázi.

Dále byly přidány rotující předměty (např. *collectible* se otáčí kolem své osy) a možnost dekorativních objektů bez kolizní detekce.

Implementované variace a jejich popis

1. Vícepatrové bludiště

Popis: Hra podporuje bludiště s více patry definovanými jako matice K x N×M. Umožňuje tvorbu volných prostor napříč patry a průlezů (*WorldElement.Hole*), kterými může hráč propadnout do nižších pater.

Realizace: Třída *World* spravuje data mapy v 3D poli *worldData*. Metoda *World.LoadMap* načítá definici pater z textového souboru. *World.GenerateWorldModels* na základě worldData generuje geometrii pro každé patro, přičemž pro *WorldElement.Hole* se nevytváří podlaha. Pád hráče skrze díry je řešen v *PlayerController* pomocí metody *world.GetGroundHeight*, která pro díru vrací float.MinValue, což způsobí pokračování pádu vlivem gravitace.

2. Minimapa

Popis: V levém horním rohu obrazovky se vykresluje zmenšená 2D mapa blízkého okolí hráče. Ukazatel hráče (šipka) je orientován k vrcholu obrazu a celá mapa se natáčí dle aktuálního otočení hráče. Minimapa zobrazuje pouze aktuální patro hráče a vizualizuje i průlezy/propadliště.

Realizace: Třída *Minimap* zodpovídá za vykreslení a využívá vlastní sadu shaderů ("*minimap.vert*", "*minimap.frag*") pro specifické 2D zobrazení. V metodě *Minimap.Render* se určuje aktuální patro hráče (*playerCurrentLevel*) a pomocí *World.GetTile* se získávají informace o prvcích mapy v daném patře pro jejich barevné odlišení. Rotace mapy a šipky je řízena horizontální rotací kamery (*camera.ry*).

3. Teleport

Popis: Hráč se může teleportovat z jednoho místa na druhé. Teleportace je doprovázena vizuálním efektem přechodu obrazu do bílé barvy a následně z bílé do pohledu na cílovou scénu.

Realizace: 2 místa pro teleportaci (*WorldElement.TeleporterPad*) jsou definována v mapě a zpracována v *World.CreateTeleportersFromMapData* na instance třídy *Teleporter*. Samotný proces teleportace a vizuální efekt řídí třída *TeleportSystem*. Tento systém používá vlastní shadery ("*fade.vert*", "*fade.frag*") pro realizaci efektu prolnutí. Metoda *TeleportSystem.StartTeleport* iniciuje fade-out efekt, během kterého *TeleportSystem.Update* postupně mění hodnotu *fadeAlpha*. *TeleportSystem.DrawFadeOverlay* vykresluje bílý překryv s danou *alphou*.
Po dosažení plného překrytí (*fadeAlpha* = 1.0) se aktualizuje pozice kamery a spustí se fade-in efekt.

4. Vícenásobné světelné zdroje

Popis: Ve scéně se podle znaků *, ^ nebo ! v mapovém souboru vytvářejí modely lamp a odpovídající světelné instance.

Realizace: V metodě *GenerateWorldModels()* třídy *World* se pro každý výskyt světelného znaku volá *CreateLightModel(),* která vytvoří model i instanci světla. Světla jsou ukládána do seznamu *worldLights*, ale *shader* aktuálně využívá pouze jedno aktivní světlo (*lightPosWorld* předávané do *Model.Draw* z *flashlight* v *Program.cs*). Vizuálně se lampy zobrazují, ale jejich světelný vliv prozatím není funkčně zapojen.

5. Otáčení objektů (animace)

Popis: Některé objekty (např. zlatá kostka – znaky 'T' v *World.GetWorldElementFromChar* vedoucí k *WorldElement.Collectible*) se otáčejí kolem své svislé osy jako jednoduchá animace.

Realizace: Objektům je při vytvoření v *World.CreateModel* nastavena hodnota *rotationSpeed*. V metodě *World.UpdateAllModels(float deltaTime)* se tato hodnota používá k plynulé rotaci modelu kolem osy Y *(model.rotation.Y)*.

6. Průchodnost a kolizní logika

Popis: Pouze určité objekty mají aktivní kolizi – například zdi (*WorldElement.Wall*). Dekorativní objekty (např. modely lamp vytvořené *CreateLightModel*, nebo sběratelské předměty *WorldElement.Collectible*) jsou průchozí.

Realizace: Modelům s kolizí je při vytvoření v *World.CreateModel* nastavena vlastnost *IsCollidable* na true a je jim přidělen AABB (Axis-Aligned Bounding Box). Při každém pohybu hráče se v *Camera.TryMove* kontroluje průnik hráčova kruhového hitboxu (*Camera.radius*) s AABB kolizních modelů (*AABB.playerIntersect*). Pokud dojde ke kolizi, testuje se pohyb zvlášť ve směru X a Z – čímž vzniká efekt klouzání podél stěn. Objekty s *IsCollidable = false* nejsou do těchto testů zahrnuty nebo test selže.

7. Neviditelné zdi (okrajová ochrana)

Popis: Pokud je na okraji mapy prázdné pole, automaticky se zde vytvoří neviditelná kolizní překážka.

Realizace: V rámci *World.GenerateWorldModels()* je při detekci volného políčka (*WorldElement.Empty*) na kraji mapy (*isEdge == true*) vytvořen model kvádru s aktivní kolizí (*IsCollidable = true*) a speciálním neviditelným materiálem (*invisibleMaterial*).

8. Přesné umístění podlahy a stropu

Popis: Každé pole má při vykreslení vlastní podlahu a strop, tvořící vizuální ohraničení pater.

Realizace: V *World.GenerateWorldModels()* se pro každou buňku (která není dírou *WorldElement.Hole*) generuje model podlahové desky (*floorMaterial*) na výšce *baseCellY - slabHeight* / 2f. Pro nejvyšší patro (nebo obecně pro ohraničení shora) se generuje stropní deska (*ceilMaterial*), například v *World.cs* je explicitně generován strop pro *level == 0* na pozici *currentStoryHeight* + *slabHeight* / 2f (relativně k nulté úrovni). Výška patra (*currentStoryHeight*) je definována na 3.0f, což odpovídá popisu "y=3" pro strop relativně k podlaze daného patra na "y=0". Modely jsou generovány pomocí *Model.CreateBlock*.

Shrnutí

Projekt realizuje 3D bludiště a demonstruje klíčové koncepty počítačové grafiky. Vychází z cvičení: implementuje transformace, práci se světlem (dynamická baterka, příprava na více zdrojů), vykreslování 3D modelů (generování geometrie *Model.CreateBlock*) a použití projekční i pohledové matice (*Camera*).

Ve scéně jsou použity materiály (*Material*), osvětlení a robustní detekce kolizí (hráč vs. objekty AABB). Implementovány jsou pokročilé funkce: vícepatrové bludiště s propadáváním, teleporty s vizuálním efektem a 2D minimapa aktuálního patra.

Struktura kódu je objektově orientovaná s typovými třídami (*Camera*, *Light*, *Model*, *World*, atd.). Projekt je připraven ke spuštění v .NET 8+ s OpenTK a je dále rozšiřitelný (např. plná podpora více světel, interaktivní prvky).